

et al. 2024), InsTag measures semantic diversity and complexity through instruction tags (Lu et al. 2023), and DEITA combines quality, complexity, and diversity (Liu et al. 2024). Other methods select for explicit diversity through quality-diversity trade-offs, log-determinant objectives, or partition-aware sampling (Bukharin et al. 2023; Wang et al. 2024; Rao et al. 2024). These methods improve data efficiency, but their rankings omit the target PEFT’s trainable parameters.

Proxy-model and influence selection. Proxy-model selectors provide reusable rankings without computing gradients for every target PEFT. Cherry LLM uses instruction-following difficulty (Li et al. 2023); Superfiltering studies when weak models can rank data for stronger models (Li et al. 2024); and SmallToLarge transfers training-trajectory information from small models to larger targets (Yang et al. 2024). Because these scores encode only example and task information, a PEFT change leaves the ranking unchanged.

Influence-based selectors model training effects more directly, but their gradients tie the score to a target parameterization. Classical influence functions use second-order approximations (Koh and Liang 2017), while TracIn accumulates gradient similarities across checkpoints (Pruthi et al. 2020). LESS builds a low-dimensional gradient datastore for instruction selection (Xia et al. 2024), and DataInf specializes influence approximation for LoRA-tuned generative models (Kwon et al. 2024). Preserving configuration specificity therefore requires rebuilding PEFT-dependent gradients when the target changes. PCU-Select instead conditions one scorer on a structured PEFT code, combining reusable sample signals with a target-specific ranking. Our evaluation compares PCU-Select with both a per-PEFT LESS implementation and an Influence baseline that uses one shared datastore.

Parameter-efficient fine-tuning. PEFT methods reduce adaptation cost in different ways, so they expose different trainable parameters. Adapters insert bottleneck modules into Transformer layers (Houlsby et al. 2019; Pfeiffer et al. 2021), while prefix and prompt tuning optimize continuous prefix states or prompts (Li and Liang 2021; Lester, Al-Rfou, and Constant 2021). LoRA learns low-rank additive updates (Hu et al. 2022), and QLoRA combines quantization with LoRA to reduce memory use (Dettmers et al. 2023). BitFit tunes bias terms (Ben Zaken, Goldberg, and Ravfogel 2022), whereas (IA)³ rescales attention and feed-forward activations (Liu et al. 2022). Prior PEFT studies usually compare accuracy, parameter count, memory, or inference overhead while holding the training data fixed. We study the complementary question: whether a fixed candidate pool has the same example ranking across PEFT configurations.

Joint data- and parameter-efficient adaptation. The closest methods use PEFT gradients for data selection, but their utility estimates remain specific to one target configuration. DataInf and LESS compute selection signals from LoRA-style training (Kwon et al. 2024; Xia et al. 2024). Geometry-aware targeted selection further shows that useful data depends on the directions available to PEFT optimization (Min et al. 2026). These methods must regenerate

configuration-dependent signals when the target changes.

PCU-Select instead describes each configuration by its intervention sites, update type, capacity, and training recipe. This representation lets one scorer reuse sample and task signals while still producing a different ranking for each configuration. The next section formalizes this requirement and the resulting cost objective.

3 Problem Formulation

We study *cross-PEFT data-efficient fine-tuning*: selecting a small training subset for a target task and PEFT configuration while reusing selection work across configurations. We fix a backbone family and a frozen base model M_0 . The candidate pool $\mathcal{D} = \{x_i\}_{i=1}^N$ contains N instruction–response examples. Each task t provides a *task sketch* V_t , a small set from its training or development data. Each PEFT configuration $p \in \mathcal{P}$ specifies a method, trainable locations, capacity, and training recipe. For a selection budget B , the ideal subset maximizes held-out task performance:

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \mathcal{D}, |\mathcal{S}| \leq B} \text{Perf}(\text{Tune}(M_0, p, \mathcal{S}), V_t^{\text{test}}). \quad (1)$$

Here Tune adapts M_0 with configuration p and subset $\mathcal{S}$, and Perf evaluates the adapted model. The held-out test set V_t^{test} defines this ideal objective but never enters training, scoring, or selection; those procedures use only V_t . Directly solving Eq. (1) would require training and evaluating many subsets for every target (p, t) pair.

PCU-Select avoids this search by assigning each example a conditional utility: its benefit for task t when training uses configuration p . The scorer s_ϕ approximates this utility for each example:

$$s_\phi(x, p, t) \approx u(x, p, t). \quad (2)$$

The target $u(x, p, t) \in [0, 1]$ summarizes task-sketch loss reductions after a few PEFT update steps. PCU-Select rank-normalizes these reductions within groups that share the same PEFT, task, and measurement setting (§4). Including p allows the ranking to change with the trainable subspace; omitting p would force every configuration to share one task-specific ranking. The task sketch V_t identifies the desired task behavior, while p identifies the updates available to the model. Utility is therefore relative to both the task and the PEFT configuration.

Amortization objective. A registry changes the cost objective because it serves multiple PEFT configurations and tasks from one candidate pool. Let P be the number of target configurations and Q the number of task sketches. LESS builds one gradient datastore for each configuration and ranks that datastore once for each task. PCU-Select instead pays one shared offline cost, followed by one scoring-and-selection pass for every PEFT–task pair:

$$\begin{aligned} C_{\text{LESS}}(P, Q) &= P(C_{\text{LESS}}^{\text{data}} + QC_{\text{LESS}}^{\text{rank}}), \\ C_{\text{PCU}}(P, Q) &= C_{\text{offline}} + PQ C_{\text{PCU}}^{\text{pair}}. \end{aligned} \quad (3)$$

Equation (3) separates four costs. $C_{\text{LESS}}^{\text{data}}$ builds one PEFT-specific datastore, and $C_{\text{LESS}}^{\text{rank}}$ ranks it for one task. C_{offline}